



LABORATORY 1

Getting started with ROS 2 and the BlueROV2

Explore a simulated ROV, discover ROS topics, and send your first motion command from Python.

Course	INRO - Instrumentation, Marine Robotics and Power Systems
Professor	Daniel Mihai Toma
Duration	One laboratory session; install software before class
Level	No previous Python or ROS 2 experience required

Mission

Find the ROV sensor and thruster topics. Observe a short dive. Then edit one Python number to make the ROV descend more gently.

You will not write a feedback controller or perform the CTD experiment today. Those are the next laboratory.

1. Prepare the computer before class

The course repository contains a Dev Container. It supplies Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic and Python. Install only the Windows tools below.

What you need

Tool	Purpose	Where to get it
Git for Windows	Clone and update the course repository.	git-scm.com/download/win
Docker Desktop	Run the Ubuntu Dev Container.	Docker Desktop for Windows
Visual Studio Code	Edit files and open container terminals.	code.visualstudio.com
Dev Containers extension	Connect VS Code to the container.	Install in the VS Code Extensions view.
GitHub access	Read the private INRO repository.	Ask the professor for access.

Windows and WSL 2

Docker Desktop uses a Linux backend. Open PowerShell as Administrator and install or update WSL if needed:

```
wsl --install
wsl --update
wsl --version
```

Restart Windows if prompted. Docker Desktop should use its WSL 2 backend. Start Docker Desktop and wait until its engine is running; verify with `docker version`. Follow Docker's current Windows requirements if your machine reports an installation problem.

Do not install ROS 2 or Gazebo directly on Windows
They come from the Dev Container, so all students use the same software versions.

2. Clone, open and launch

Clone the repository once

Open PowerShell in a normal local folder, such as Documents. Sign in to GitHub when prompted for the private repository.

```
cd $HOME\Documents
mkdir GitHub
cd GitHub
git clone https://github.com/danielMihaiToma/INRO-Marine-Robotics.git
cd INRO-Marine-Robotics
code .
```

Open the Dev Container

- 1 Start Docker Desktop and wait for the engine.
- 2 In VS Code, open the INRO-Marine-Robotics folder.
- 3 Press Ctrl+Shift+P and select Dev Containers: Reopen in Container.
- 4 Wait for the initial container build. Open a new VS Code terminal after the status bar says Dev Container.

Already inside?

If VS Code offers Reopen Folder Locally, you are already connected to the container.

Build and launch the ROV

Run these commands in a container terminal. Keep the final command running.

```
cd /workspaces/INRO-Marine-Robotics/ros2_ws
python3 src/inro_rov_demo/scripts/prepare_model.py
colcon build --packages-select inro_rov_demo --symlink-install
source install/setup.bash
ros2 launch inro_rov_demo demo.launch.py
```

The model download and first build can take longer. The simulator starts in a small pool: the ROV is near 2 m depth and the floor is near 5 m.

Camera control

Right-click bluerov2 in Gazebo, choose Move to, zoom with the mouse wheel, then choose Follow. Follow does not zoom automatically.

3. Discover ROS nodes and topics

A ROS node is a running program. A topic is a named stream of messages. Gazebo, the bridge and the thruster mixer communicate through these streams.

The idea

Gazebo ROV → ROS bridge → sensor topics → your terminal

Your Python command → /inro/command → thruster mixer → six thruster-force topics → Gazebo ROV

Open a second terminal inside the same Dev Container

```
cd /workspaces/INRO-Marine-Robotics/ros2_ws
source install/setup.bash
ros2 node list
ros2 topic list -t
ros2 topic info /inro/command
```

Use the output to complete this worksheet. Topic names are case-sensitive.

Find a topic that...	Topic name	Message type
reports depth		
reports position/velocity		
reports IMU readings		
receives a vehicle command		
sends force to one thruster		

The six thruster topics are /inro/thruster_1/force through /inro/thruster_6/force. There is no single /inro/thrusters topic.

Questions

How many /inro/ topics can you find? Which node appears to convert one vehicle command into individual thruster commands?

4. Observe a short movement

In the second terminal, display the ROV depth. Stop the stream with Ctrl+C when you have enough data.

```
ros2 topic echo /inro/depth
```

Open a third terminal, run `source install/setup.bash`, and send a timed downward command:

```
ros2 run inro_rov_demo drive down --seconds 4 --power 0.4
```

Depth is measured in metres and increases downward. Restart the simulator between trials if you want to compare from the same starting pose.

Explore sensors and thrusters

Use `ros2 topic echo TOPIC_NAME` to inspect `/inro/odometry`, `/inro/imu`, and `/inro/thruster_5/force` or `/inro/thruster_6/force`. You do not need to understand every field yet. Then try:

```
ros2 run inro_rov_demo drive forward --seconds 3 --power 0.4
ros2 run inro_rov_demo drive left --seconds 3 --power 0.4
```

Run these separately. Observe which thruster forces change and what the ROV does.

Motion	Thrusters that change	What did the ROV do?
Down		
Forward		
Left turn		
Command finished		

Important observation

A zero command stops thrust; it does not instantly stop motion or hold depth. Inertia and buoyancy can still move the ROV. An automatic depth hold requires feedback control in Laboratory 2.

5. Your first Python command

The course provides the working ROS 2 program. You only edit one number.

```
ros2_ws/src/inro_rov_demo/lab01/first_dive.py
```

Find this line near the top:

```
HEAVE_EFFORT = -0.40
```

The program repeatedly publishes [surge, heave, yaw] to /inro/command for four seconds. Here only the middle value changes. Negative heave requests downward thrust; positive heave requests upward thrust.

Value	Prediction before running	Observation after running
-0.40		
-0.20		
+0.20 (optional)		

Run the script

Keep `demo.launch.py` running. In another sourced container terminal:

```
cd /workspaces/INRO-Marine-Robotics/ros2_ws
source install/setup.bash
python3 src/inro_rov_demo/lab01/first_dive.py
```

- 1 Predict the motion with -0.40. Run the script and record depth before and after.
- 2 Restart the simulator. Change the one line to -0.20 and repeat.
- 3 Optional: predict and try +0.20. Keep the effort magnitude at or below 0.65.

Effort is not speed

The values -0.40 and -0.20 are normalized thrust requests, not speeds in m/s. The achieved speed depends on drag, inertia and buoyancy. Controlling a requested speed is a Laboratory 2 task.

6. Submit and look ahead

What to submit

- Your edited `first_dive.py`.
- The ROS topic worksheet from page 4.
- The motion-and-thruster table from page 5.
- Depth before and after two Python trials, plus two or three sentences explaining the sign and magnitude of heave effort.

Troubleshooting

Problem	Check
No "Reopen in Container" option	Is Docker Desktop running? Is the Microsoft Dev Containers extension installed? If VS Code offers "Reopen Folder Locally," you are already inside.
ROS package not found	Build from <code>ros2_ws</code> , then source <code>install/setup.bash</code> in each new container terminal.
No Gazebo window	Check the launch terminal for errors and confirm that your Windows WSLg GUI support is working.
Python reports no mixer	Start <code>demo.launch.py</code> first, and run only one copy of the simulator.
ROV becomes hard to see	Right-click <code>bluerov2</code> → Move to, zoom with the wheel, then Follow.
Depth changes after command ends	This is expected: zero thrust is not depth hold.

Next laboratory

Laboratory 2: BlueROV2 depth control

You will use the topics you discovered today to write a feedback controller, descend to 20 m and 50 m, hold each depth, and study how a CTD density profile changes buoyancy.

References

- [Docker Desktop installation for Windows](#)
- [VS Code Dev Containers guide](#)
- [GitHub cloning guide](#)
- [ROS 2 Jazzy tutorials](#)

Use these commands only with the simulation, never with a real ROV.